

Determining What Strategy a Pitcher Employs: Linear Algebra and Baseball

Simon Christensen

May 15, 2026

Abstract

Although there is an abundance of strategies that a pitcher can employ to induce an out on the baseball field, the following research will only look at two: pitch velocity and break. To stay consistent, break will be measured as the average distance, in inches, from the center of the strike zone. These two strategies are the easiest to see visibly while watching baseball and most likely the easiest to understand in terms of being ways pitchers can be successful. These two strategies will be studied using the Velocity Dominance Index (VDI) and the Break Dominance Index (BDI), and these metrics will further be used to determine with what pitches each pitcher is most dominant with and why they are dominant with those pitches. Another measurement that will be involved in determining how successful a pitcher is at employing a particular strategy is whiff rate, or the number of times a hitter swung and missed at a pitch divided by the total number of swings they took.

1 Introduction

The two pitchers being studied will be Corbin Burnes, starting pitcher for the Arizona Diamondbacks, and Aroldis Chapman, relief/closing pitcher for the Boston Red Sox. Burnes utilizes a five pitch arsenal consisting of a four-seam fastball (FF), cutter (C), sinker (S), slider (SL), curveball (CB). Burnes had his breakout year in 2021, when he threw a no hitter and broke the record for the most strikeouts in a row without issuing a walk. At the end of that season, he won the Cy Young award and today he is a four-time all star. Research shows that Burnes has movement on all of his pitches while still having decent velocity on his fastball pitches (FF,C,S). Chapman, on the other hand, is a flame-throwing left-handed pitcher who utilizes a four-pitch arsenal consisting of a four-seam fastball, split-finger (SP), sinker, and slider. He holds the record for the fastest pitch ever thrown at 105.8 miles per hour. He also is the all-time leader for the most strikeouts by a left-handed reliever and a member of the 300 save club while having won two world series championships.

Now to show what strategies these pitchers use, I'll first show how the data was collected. I used average velocity, usage (in percentage), vertical break (VB), horizontal break (HB), and whiff rate (WR) for each pitch per year for each pitcher. Each set of data was compiled into a five-by-five matrix for Burnes since he has five pitches and a four-by-four matrix for

Chapman since he has four pitches. Using the matrices compiled with average velocities and usage, I calculated a new matrix representing the VDI. Using the matrices compiled with average vertical and horizontal break, I calculated the BDI. However what is the Velocity and Break Dominance Indexes? VDI is a crucial measurment for pitchers and shows how a pitcher's velocity changes over each season proportional to its usage. For example, a pitch with a high velocity and high usage would have a high VDI value. Moreover, BDI takes into account horizontal and vertical break and measures how a pitcher's movement on each of his pitches changes every season proportional to its usage. The formulae for these two indexes are the following:

$$VDI_i = \sum_{j=1}^n P_{ji} V_{ji} \quad \text{and} \quad BDI_i = \sum_{j=1}^n P_{ji} \sqrt{V B_{ji}^2 + H B_{ji}^2}$$

These two equations produce the following two matrices which represent the VDI and BDI for Corbin Burnes for each pitch at each year, starting at 2021.

VDI Matrix: Corbin Burnes						BDI Matrix: Corbin Burnes					
	FF	C	S	SL	CB		FF	C	S	SL	CB
2021	1.444	49.790	8.906	8.175	14.778	2021	0.230	6.256	1.548	0.726	2.509
2022	0.480	52.630	6.445	8.114	15.014	2022	0.067	6.770	1.129	0.690	2.164
2023	0.480	52.298	7.338	7.310	13.646	2023	0.070	6.438	1.176	0.606	2.576
2024	0.668	43.552	7.760	12.731	17.264	2024	0.100	5.817	1.277	1.222	3.509
2025	0.193	51.661	9.178	7.911	15.360	2025	0.029	6.826	1.599	0.581	2.889

Next we will do the same for Chapman, starting at 2022 since he only has four pitches.

VDI Matrix: Aroldis Chapman					BDI Matrix: Aroldis Chapman				
	FF	SP	S	SL		FF	SP	S	SL
2022	55.638	9.812	5.734	22.722	2022	10.447	0.249	1.148	3.079
2023	54.113	13.127	5.010	21.082	2023	10.257	0.559	0.992	2.511
2024	47.322	6.418	16.378	25.461	2024	8.488	0.377	3.079	3.175
2025	33.545	10.992	26.973	23.229	2025	6.452	0.785	5.482	3.023

These four matrices will be used to conduct further research on the strategies the pitchers use and the pitches they use to implement their strategy.

2 Method

Finding eigenvalues and eigenvectors of each VDI and BDI matrix for each pitcher is fundamental in narrowing in on how each pitcher's pitching style varied throughout each year. Large eigenvalues denote a strong pattern of variation (deception) across years, while a small (or zero) eigenvalue exhibits little or no variation in a pitchers velocity or break, depending on the matrix from which the eigenvalue is from. Eigenvectors can determine the pitch that drives most of the variation, depending on the position in which the largest component is

in. Before calculating, I first normalized each VDI and BDI matrix by dividing each matrix by its own Frobenius norm (see Appendix-4). This makes it easier to compare the pitchers' data as the only thing about each matrix that will change is its size. We can also calculate and observe the norms of each row and column of each VDI and BDI matrix (see Appendix-5). However, when it comes to numerical analysis, the norms won't tell much of a story compared to the eigenvalues and eigenvectors since norms only relate to the magnitude of vectors and are just a single-number summary of a vector. We can easily determine the magnitude of each vector, (each row and column), just by looking at each matrix.

3 Numerical Analysis

The major question here is, how do the eigenvalues and eigenvectors relate to what strategy a pitcher is using? Firstly, I want to make some assumptions based on what I previously stated about how eigenvalues and eigenvectors will show the behavior of a pitcher's strategy. To start with, large eigenvalues will push modes more in a particular direction than smaller ones. A mode is a combination of pitch types that evolve together. Each mode corresponds to one eigenvalue (how the pattern changes over time) and its corresponding eigenvector (the pattern itself). Modes can be used to show the eigendirections of the data I procured for each pair of eigenvalues/vectors. Another assumption I want to make is that imaginary eigenvalues or imaginary components of eigenvectors can exhibit cyclic behavior between strategies. For example, maybe one year, Corbin Burnes relies more on his cutter than his other pitches. As his use of his cutter increases or decreases over each year, the use of his other pitches must also increase or decrease, hence causing a cycle to occur between strategies. Now to see what is going on underneath the hood, I will show in depth the methods I used on the VDI matrix of Corbin Burnes and provide an ordered list of the methods I used to dive deep into the numerical analysis of each pitchers' strategy. The same methods will be used on all four matrices.

- i. **Matrix Construction:** (VDI and BDI)
- ii. **Linear Evolution:** Assume that each pitch will evolve approximately linearly from year to year, which looks like $x(t+1) \approx Ax(t)$ where x is each column of the VDI matrix. From here, we need to solve for A which we can do using this equation: $X_2 = AX_1$, where
$$X_1 = \begin{bmatrix} x(1) & x(2) & \cdots & x(t-1) \end{bmatrix}, \quad X_2 = \begin{bmatrix} x(2) & x(3) & \cdots & x(t) \end{bmatrix}.$$

Since X_1 may be singular or not square, (in my case X_1^{-1} does exist), singular value decomposition (SVD) allows us to use the pseudo inverse of X_1 to give us the best A possible.
- iii. **Eigenvalues/vectors:** $Av_i = \lambda_i v_i$ produces λ_i , the eigenvalue of mode i , and v_i , the eigenvector for the same mode.
- iv. **Modal Matrix:** Construct

$$V = \begin{bmatrix} v_1 & v_2 & \cdots & v_m \end{bmatrix}.$$

v. **Computing modal amplitudes b** ; The initial strategy snapshot can be written as

$$x(1) = \sum_{i=1}^m b_i v_i.$$

which implies

$$b = V^{-1}x(1).$$

vi. **Time Evolution of Each Mode**: each mode will evolve according to

$$mode_i(i) = b_i \lambda_i^t.$$

This gives each mode in terms of its amplitude and phase.

vii. **Contribution per pitch for a Given Mode**: For pitch j ,

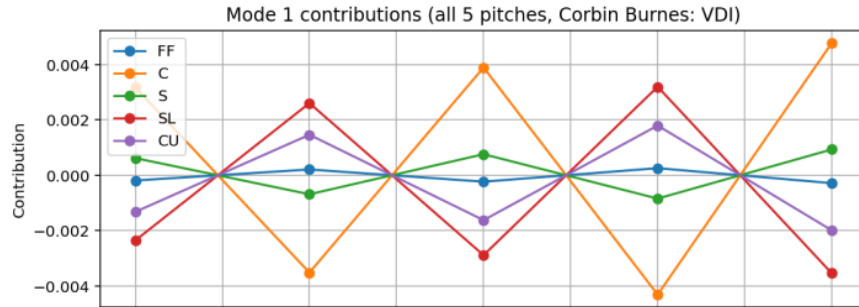
$$contribution_{i,j}(t) = Re(b_i \lambda_i^t v_i).$$

This produces a matrix of values that can tell the full story of how every other pitch from pitch j is dependent on pitch j . In other words, we will be able to study how the behaviors of strategies are dependent on each pitch being used.

Mode 1:

	FF	C	S	SL	CB
2021	-0.000189	0.003175	0.00062	-0.002355	-0.001322
2022	0.000209	-0.003517	-0.000687	0.002608	0.001465
2023	-0.000232	0.003896	0.000761	-0.002889	-0.001622
2024	0.000257	-0.004315	-0.000843	0.003199	0.001797
2025	-0.000285	0.004779	0.000934	-0.003544	-0.00199

Graphing each column over each year, we can get a visual for what pitches Burnes throws together based off of the velocity of each pitch since we used the VDI matrix to obtain this data. Below is the graph for mode 1 of Burnes' VDI matrix:



4 Discussion

In total, the data will provide 18 graphs; 5 for Burnes' VDI and BDI matrix and 4 for Chapman's respective matrices. Now what's happening in the above graph and what does it tell us about Corbin Burnes' pitching strategies? We should remember that this graph shows mode 1 for Burnes' VDI graph, so this means that this graph shows his tendencies to throw his five-pitch mix based on the velocity of each pitch. Also, since these modes are basically approximating what strategies each pitcher actually used, note that mode 1 for each pitchers' matrices will be the most telling as those modes use the largest eigenvalue and therefore will take into consideration the biggest variation between strategies. From this mode, we can extrapolate that his usage of his cutter and slider oscillate opposite from each other year to year. This effect also occurs between his curveball and his sinker with less magnitude, all while his fastball usage stays consistent. To see if this graph is consistent with Burnes' actual strategy for years 2021 through 2025, we'll dive into some relatively simple baseball pitching techniques. Note that a cutter has slight glove-side run, similar to sliders and curveballs, however sliders and curveballs have much more break than a cutter. On the other hand, four-seam fastballs have a natural arm-side run and are similar to sinkers except a sinker's run has greater magnitude. The graph shows that Burnes' likes to pair his cutter with his sinker and his slider with his curveball, which makes intuitive sense. Pitchers nowadays use a fantastic technique called pitch tunneling, which refers to pitchers throwing pitches that look the same out of the hand but end up in different parts of the strike zone. Considering the cutter/sinker combination, cutters have glove-side movement while sinkers move arm-side. In order to fool hitters, Burnes alternates between his cutter and sinker as these pitches occupy the same "tunnel" but the cutter will normally end up on the left side of the plate relative to the pitcher and the sinker on the right. Then later in at bats, he utilizes his slider and curveball to achieve a swing and miss (whiff) because hitters will think that he is throwing his cutter or sinker, which both have considerably less break than a slider or curveball. This is readily apparent, as both his slider and curveball have a whiff rate between 40-50% between 2021 and 2025. Therefore if Corbin Burnes can execute good break on his cutter and sinker as well as place them well enough in the strike zone such that they appear the same to the hitter out of the hand, he can resort to his slider and curveball to induce weak contact or even strikeouts. One interesting note to make is that these oscillations between pitch mixes occur in this mode due to imaginary components in the eigenvector associated with this mode. This property of imaginary numbers shows up everywhere in mathematics, from dynamical systems involving population growth and decay to fluid dynamics. It is nice to see the consistency of these crucial mathematical structures beautifully hidden in the sport of baseball. Getting back on track, note that even though the whiff rate of his sinker decreased to below 5%, his slider's whiff rate slightly increased, implying Burnes utilized the tunneling of his sinker and slider effectively, (See Appendix-3 for whiff rate data). From all of the analysis above, we can firmly conclude that Burnes employs pitch break to achieve outs. He uses the incredible break on his cutter, sinker, slider, and curveball to fool hitters, and doing so at an elite level.

Ultimately the same analysis can be done for Aroldis Chapman, so let's dive into how Chapman's strategies behave, (refer to Appendix-6 for mode graphs). Looking at the mode 1 graph for Chapman's VDI matrix, we see how his four-pitch mix fans out as the years

progress. His sinker increases, his split-finger and slider stay relatively consistent, and his fastball decreases. This also makes intuitive sense, and here is why. According to statistics pulled from MLB Statcast, the number of pitches thrown above 100 miles per hour in the early 2000's was below 500. Now in this current decade, that number is now in the couple thousands, and steadily rising as more teams acquire more pitchers who have extreme velocity. This does come with drawbacks due to the fact that as more and more hitters face pitchers who throw 100+ mph, they will be able to time up and hit even the fastest of pitches. How does this relate to Chapman? Throughout the 2022 through 2025 seasons, Chapman began to adopt a sinker which is crucial in today's game since pitchers need more than just velocity to get hitters out. Not only is the sinker a type of fastball pitch and therefore can be thrown at high speeds, sinkers can have a massive amount of arm-side run if thrown properly. That is exactly what Chapman uses to his advantage and that is what the mode 1 graph shows. A steady rise in the usage of his sinker has led Chapman to be the most feared relief/closing pitcher in the MLB. Although it is bad enough that he is able to reach speeds of 105 mph, he also has two strikeout pitches in his slider and split-finger, which break on opposite sides of the plate, to compliment his extreme velocity. His slider and split-finger also allow Chapman to tunnel his pitches just as discussed with Corbin Burnes, if he wasn't in the advantage already. To conclude analysis on Aroldis Chapman, even though he has incredible break on his two off-speed pitches, his velocity is what gives him that extra edge when on the mound.

In conclusion, every successful pitcher must utilize some amount of both pitch velocity and break when on the mound as hitters are too skilled to just use one or the other to try and get outs. However, different pitchers have different tendencies to throw more firm pitches that can blow by hitters or softer-velocity pitches that can fool hitters with break and difference in speed. I think it is better to say that the best pitchers compliment each strategy with the other using pitch tunneling to ensure that the hitters never know what's coming. But for the sake of the argument, based on the data and analyzing of said data, pitchers employ velocity and break in unique ways in order to be successful and stay one step ahead of hitters..

References

- [1] FanGraphs Staff. *FanGraphs: The most comprehensive fan index in baseball*. Available at: <https://www.fangraphs.com/>. Accessed: November 15, 2025.
- [2] MLB Advanced Media. *Baseball Savant: Statcast Search and Player Tracking*. Available at: <https://baseballsavant.mlb.com/>. Accessed: November 15, 2025.
- [3] TopVelocity Performance Labs. *The Critical Role of Velocity in Pitching*. Available at: <https://www.topvelocity.net/2024/05/06/the-critical-role-of-velocity-in-pitching/>. Published: 2024. Accessed: November 15, 2025.
- [4] Columbia SABR Lions. *Inside the TrackMan, Part I: The Non-Linear Impact of Velocity on Mid-Major Pitching Suc-*

cess. Available at: <https://medium.com/columbia-sabr-lions/inside-the-trackman-part-i-the-non-linear-impact-of-velocity-on-mid-major-pitching-s>
Accessed: November 15, 2025.

- [5] Baseball America Staff. *Why Do MLB Pitchers Focus So Much On Fastball Velocity? How Fastball Data Explains Baseball's Growing Search for Speed*. Baseball America, 2024. Available at: <https://www.baseballamerica.com/stories/why-do-mlb-pitchers-focus-so-much-on-velocity-how-fastball-data-explains-baseballs-g>
Accessed: November 15, 2025.

- [6] Marie-Andrée Mercier, Mathieu Tremblay, Catherine Daneau, and Martin Descarreaux. *Individual Factors Associated with Baseball Pitching Performance: Scoping Review*. BMJ Open Sport & Exercise Medicine, 6(1):e000704, 2020. Available at: <https://bmjopensem.bmj.com/content/6/1/e000704>. Accessed: November 15, 2025.

5 Appendix

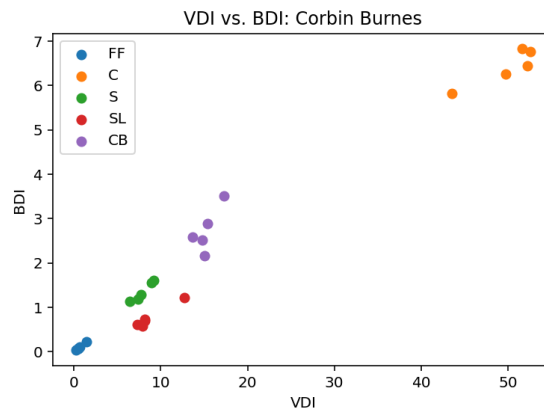
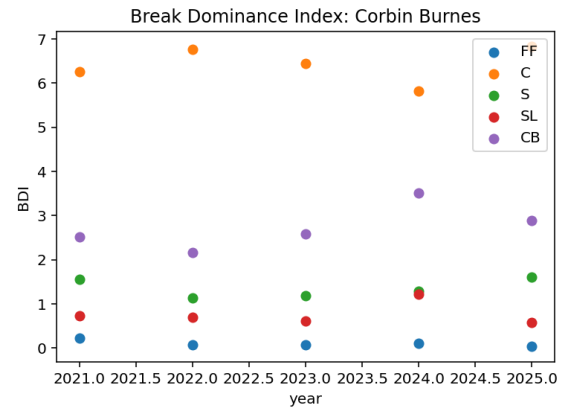
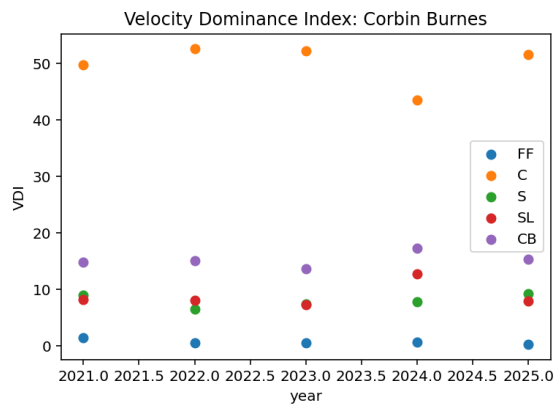
1) Corbin Burnes' data

```
1
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 # Corbin Burnes data
7
8 V_cb=np.array([[96.3,95.2,96.8,87.9,81.2], # mph
9               [96.1,95,96.2,88.2,81.6],
10              [96,94.4,95.3,86,79.8],
11              [95.4,95.3,97,87.2,80.3],
12              [96.5,94.1,95.6,87.9,80]])
13 P_cb=.01*np.array([[1.5,52.3,9.2,9.3,18.2], # percentage
14                   [0.5,55.4,6.7,9.2,18.4],
15                   [0.5,55.4,7.7,8.5,17.1],
16                   [0.7,45.7,8,14.6,21.5],
17                   [0.2,54.9,9.6,9,19.2]])
18 VDI_cb=P_cb*V_cb
19 HB_cb=np.array([[ -5.5,4.2,-11.8,7.8,11.3], # inches,
20                negative=arm-side distance
21                [-3.5,4.4,-11.3,7.5,9.8], # from center of the
22                strike zone
23                [-4,3.1,-10.7,7.1,10.4],
24                [-3,2.4,-12.2,8.3,10],
25                [-4.2,2.4,-12.6,6.3,9.7]])
26 VB_cb=np.array([[14.3,11.2,12,-0.3,-7.9], # inches,
27                negative=distance below the
```

```

25         [13,11.4,12.5,-0.1,-6.5],      # center of the strike
26         zone
27         [13.5,11.2,10.9,0.7,-10.9],
28         [13.9,12.5,10.3,-1.1,-12.9],
29         [14.1,12.2,10.9,1.4,-11.5]])
30 # VDI/BDI: Corbin Burnes
31
32 VDI_cb=P_cb*V_cb                                # VDI: Corbin Burnes
33 BDI_cb=P_cb*np.sqrt(HB_cb**2 + VB_cb**2)        # BDI: Corbin Burnes
34 year1=[2021,2022,2023,2024,2025]
35 year2=[2022,2023,2024,2025]
36 plt.figure()
37 plt.scatter(year1,VDI_cb[:,0],label='FF')
38 plt.scatter(year1,VDI_cb[:,1],label='C')
39 plt.scatter(year1,VDI_cb[:,2],label='S')
40 plt.scatter(year1,VDI_cb[:,3],label='SL')
41 plt.scatter(year1,VDI_cb[:,4],label='CB')
42 plt.xlabel('year')
43 plt.ylabel('VDI')
44 plt.legend()
45 plt.title('Velocity Dominance Index: Corbin Burnes')
46 plt.figure()
47 plt.scatter(year1,BDI_cb[:,0],label='FF')
48 plt.scatter(year1,BDI_cb[:,1],label='C')
49 plt.scatter(year1,BDI_cb[:,2],label='S')
50 plt.scatter(year1,BDI_cb[:,3],label='SL')
51 plt.scatter(year1,BDI_cb[:,4],label='CB')
52 plt.title('Break Dominance Index: Corbin Burnes')
53 plt.xlabel('year')
54 plt.ylabel('BDI')
55 plt.legend()
56 plt.figure()
57 plt.scatter(VDI_cb[:,0],BDI_cb[:,0],label='FF')
58 plt.scatter(VDI_cb[:,1],BDI_cb[:,1],label='C')
59 plt.scatter(VDI_cb[:,2],BDI_cb[:,2],label='S')
60 plt.scatter(VDI_cb[:,3],BDI_cb[:,3],label='SL')
61 plt.scatter(VDI_cb[:,4],BDI_cb[:,4],label='CB')
62 plt.title('VDI vs. BDI: Corbin Burnes')
63 plt.legend()
64 plt.xlabel('VDI')
65 plt.ylabel('BDI')

```

2) Aroldis Chapman's data

```

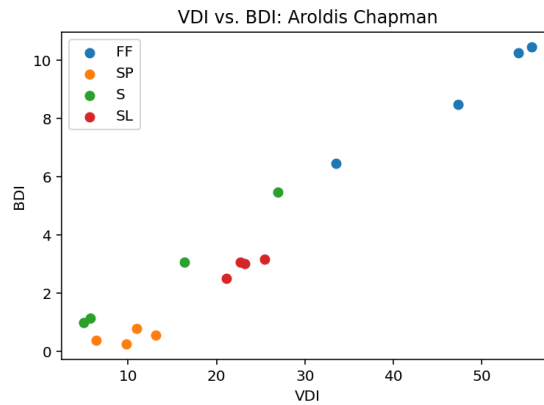
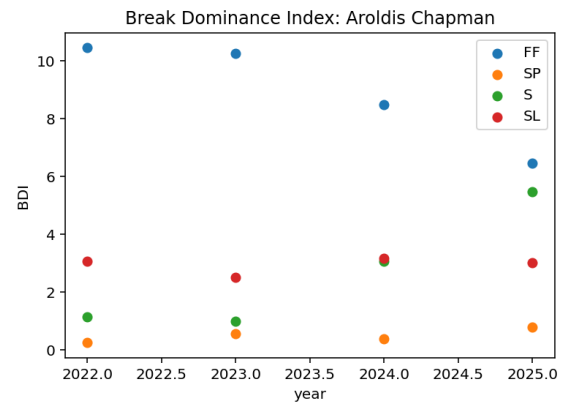
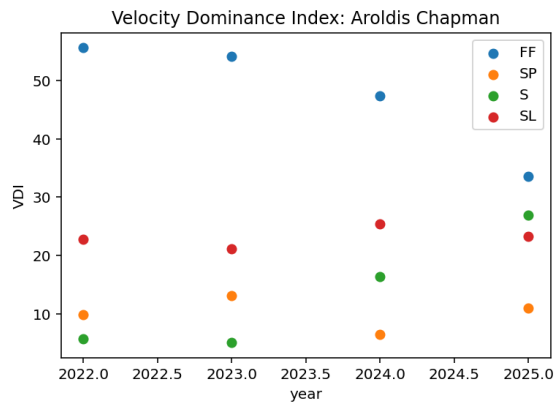
1 # Aroldis Chapman data
2
3 V_ac=np.array([[98.3,88.4,100.6,85.1],      # mph
4               [97.5,88.1,100.2,85.7],
5               [99,90.4,101.1,88.1],
6               [97.8,91.6,99.9,87]])
7 P_ac=.01*np.array([[56.6,11.1,5.7,26.7],    # percentage
8                   [55.5,14.9,5,24.6],
9                   [47.8,7.1,16.2,28.9],
10                  [34.3,12,27,26.7]])
11 VDI_ac=P_ac*V_ac
12 HB_ac=np.array([[ -4.5, -2.1, -12.5, 10.7],  # inches,
13                negative=arm-side distance
14                [ -4.6, -2.7, -12.4, 8.4],    # from center of the
15                strike zone
16                [ -4, -4, -11, 9],
17                [ -3.4, -5.1, -11.1, 10.3]])
18 VB_ac=np.array([[17.9, -0.8, 15.8, 4.3],     # inches,
19                negative=distance below the
20                [17.9, 2.6, 15.5, 5.8],       # center of the strike
21                zone
22                [17.3, 3.5, 15.5, 6.3],
23                [18.5, 4.1, 17, 4.7]])
24
25 # VDI/BDI: Aroldis Chapman
26
27 VDI_ac=P_ac*V_ac                                # VDI: Aroldis Chapman
28 BDI_ac=P_ac*np.sqrt(HB_ac**2 + VB_ac**2)        # BDI: Corbin Burnes
29
30 plt.figure()
31 plt.scatter(year2,VDI_ac[:,0],label='FF')
32 plt.scatter(year2,VDI_ac[:,1],label='SP')
33 plt.scatter(year2,VDI_ac[:,2],label='S')
34 plt.scatter(year2,VDI_ac[:,3],label='SL')
35 plt.xlabel('year')
36 plt.ylabel('VDI')
37 plt.legend()
38 plt.title('Velocity Dominance Index: Aroldis Chapman')
39 plt.figure()
40 plt.scatter(year2,BDI_ac[:,0],label='FF')
41 plt.scatter(year2,BDI_ac[:,1],label='SP')
42 plt.scatter(year2,BDI_ac[:,2],label='S')
43 plt.scatter(year2,BDI_ac[:,3],label='SL')
44 plt.xlabel('year')
45 plt.ylabel('BDI')
46 plt.legend()

```

```

43 plt.title('Break Dominance Index: Aroldis Chapman')
44 plt.figure()
45 plt.scatter(VDI_ac[:,0],BDI_ac[:,0],label='FF')
46 plt.scatter(VDI_ac[:,1],BDI_ac[:,1],label='SP')
47 plt.scatter(VDI_ac[:,2],BDI_ac[:,2],label='S')
48 plt.scatter(VDI_ac[:,3],BDI_ac[:,3],label='SL')
49 plt.title('VDI vs. BDI: Aroldis Chapman')
50 plt.legend()
51 plt.xlabel('VDI')
52 plt.ylabel('BDI')

```



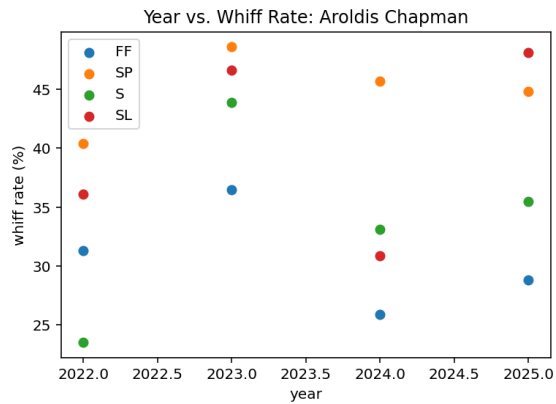
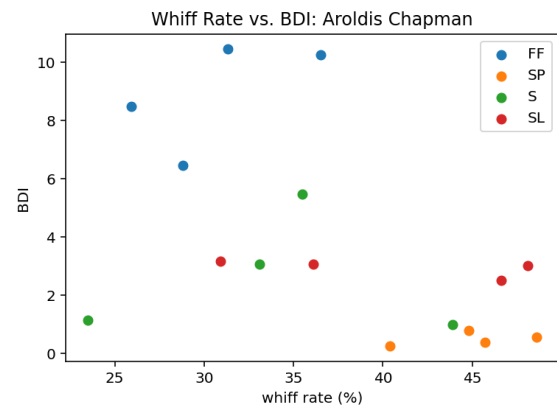
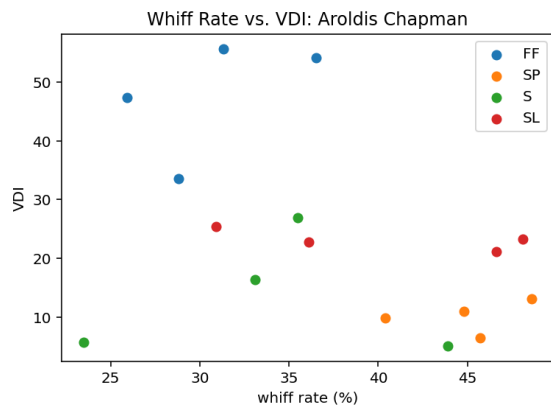
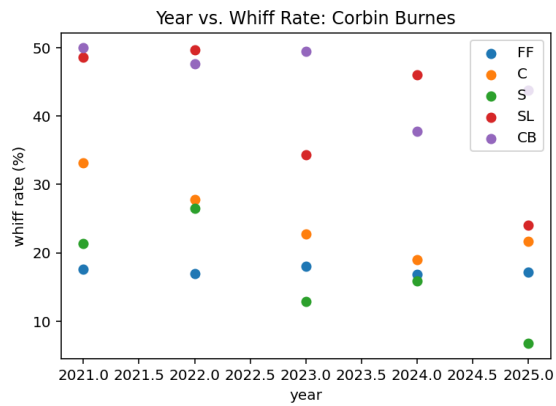
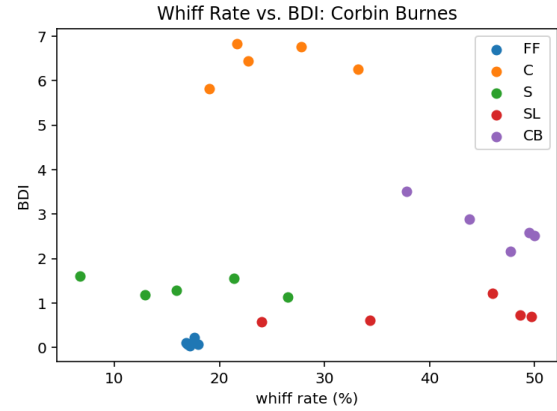
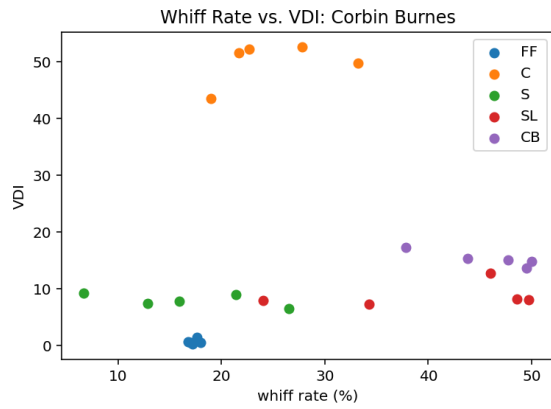
3) Whiff Rate data

```
1 wr_cb=np.array([[17.6,33.2,21.4,48.6,50],
2                [17,27.8,26.5,49.7,47.7],
3                [18,22.7,12.9,34.3,49.5],
4                [16.8,19,15.9,46,37.8],
5                [17.2,21.7,6.7,24,43.8]])
6 wr_ac=np.array([[31.3,40.4,23.5,36.1],
7                [36.5,48.6,43.9,46.6],
8                [25.9,45.7,33.1,30.9],
9                [28.8,44.8,35.5,48.1]])
10
11 # Whiff Rate Graphs
12
13 plt.figure()
14 plt.scatter(wr_cb[:,0],VDI_cb[:,0],label='FF')
15 plt.scatter(wr_cb[:,1],VDI_cb[:,1],label='C')
16 plt.scatter(wr_cb[:,2],VDI_cb[:,2],label='S')
17 plt.scatter(wr_cb[:,3],VDI_cb[:,3],label='SL')
18 plt.scatter(wr_cb[:,4],VDI_cb[:,4],label='CB')
19 plt.title('Whiff Rate vs. VDI: Corbin Burnes')
20 plt.xlabel('whiff rate (%)')
21 plt.ylabel('VDI')
22 plt.legend()
23 plt.figure()
24 plt.scatter(wr_cb[:,0],BDI_cb[:,0],label='FF')
25 plt.scatter(wr_cb[:,1],BDI_cb[:,1],label='C')
26 plt.scatter(wr_cb[:,2],BDI_cb[:,2],label='S')
27 plt.scatter(wr_cb[:,3],BDI_cb[:,3],label='SL')
28 plt.scatter(wr_cb[:,4],BDI_cb[:,4],label='CB')
29 plt.title('Whiff Rate vs. BDI: Corbin Burnes')
30 plt.xlabel('whiff rate (%)')
31 plt.ylabel('BDI')
32 plt.legend()
33 plt.figure()
34 plt.scatter(year1,wr_cb[:,0],label='FF')
35 plt.scatter(year1,wr_cb[:,1],label='C')
36 plt.scatter(year1,wr_cb[:,2],label='S')
37 plt.scatter(year1,wr_cb[:,3],label='SL')
38 plt.scatter(year1,wr_cb[:,4],label='CB')
39 plt.title('Year vs. Whiff Rate: Corbin Burnes')
40 plt.xlabel('year')
41 plt.ylabel('whiff rate (%)')
42 plt.legend()
43 plt.figure()
44 plt.scatter(wr_ac[:,0],VDI_ac[:,0],label='FF')
45 plt.scatter(wr_ac[:,1],VDI_ac[:,1],label='SP')
46 plt.scatter(wr_ac[:,2],VDI_ac[:,2],label='S')
```

```

47 plt.scatter(wr_ac[:,3],VDI_ac[:,3],label='SL')
48 plt.title('Whiff Rate vs. VDI: Aroldis Chapman')
49 plt.xlabel('whiff rate (%)')
50 plt.ylabel('VDI')
51 plt.legend()
52 plt.figure()
53 plt.scatter(wr_ac[:,0],BDI_ac[:,0],label='FF')
54 plt.scatter(wr_ac[:,1],BDI_ac[:,1],label='SP')
55 plt.scatter(wr_ac[:,2],BDI_ac[:,2],label='S')
56 plt.scatter(wr_ac[:,3],BDI_ac[:,3],label='SL')
57 plt.title('Whiff Rate vs. BDI: Aroldis Chapman')
58 plt.xlabel('whiff rate (%)')
59 plt.ylabel('BDI')
60 plt.legend()
61 plt.figure()
62 plt.scatter(year2,wr_ac[:,0],label='FF')
63 plt.scatter(year2,wr_ac[:,1],label='SP')
64 plt.scatter(year2,wr_ac[:,2],label='S')
65 plt.scatter(year2,wr_ac[:,3],label='SL')
66 plt.title('Year vs. Whiff Rate: Aroldis Chapman')
67 plt.xlabel('year')
68 plt.ylabel('whiff rate (%)')
69 plt.legend()

```



4) Eigenvalues/Eigenvectors of VDI/BDI

```

1 # Eigenvalues/Eigenvectors of VDI/BDI
2
3 eVcb,vVcb=np.linalg.eig(VDI_cb)
4 eBcb,vBcb=np.linalg.eig(BDI_cb)
5 eVac,vVac=np.linalg.eig(VDI_ac)
6 eBac,vBac=np.linalg.eig(BDI_ac)
7 AVcb=np.zeros((5,6),dtype=complex)
8 AVcb[:,0]=eVcb
9 AVcb[:,1]=vVcb[0]
10 AVcb[:,2]=vVcb[1]
11 AVcb[:,3]=vVcb[2]
12 AVcb[:,4]=vVcb[3]
13 AVcb[:,5]=vVcb[4]
14 AVcb=np.round(AVcb.real,2)+1j*np.round(AVcb.imag,2)
15 t1=pd.DataFrame(AVcb,index=['|','|','|','|','|'],
16                  columns=['Eigenvalues','v1','v2','v3','v4','v5'])
17 print(t1)
18 ABcb=np.zeros((5,6),dtype=complex)
19 ABcb[:,0]=eBcb
20 ABcb[:,1]=vBcb[0]
21 ABcb[:,2]=vBcb[1]
22 ABcb[:,3]=vBcb[2]
23 ABcb[:,4]=vBcb[3]
24 ABcb[:,5]=vBcb[4]
25 ABcb=np.round(ABcb.real,2)+1j*np.round(ABcb.imag,2)
26 t2=pd.DataFrame(ABcb,index=['|','|','|','|','|'],
27                  columns=['Eigenvalues','v1','v2','v3','v4','v5'])
28 print(t2)
29 AVac=np.zeros((4,5))
30 AVac[:,0]=eVac
31 AVac[:,1]=vVac[0]
32 AVac[:,2]=vVac[1]
33 AVac[:,3]=vVac[2]
34 AVac[:,4]=vVac[3]
35 t3=pd.DataFrame(AVac,index=['|','|','|','|'],
36                  columns=['Eigenvalues','v1','v2','v3','v4'])
37 print(t3)
38 ABac=np.zeros((4,5))
39 ABac[:,0]=eBac
40 ABac[:,1]=vBac[0]
41 ABac[:,2]=vBac[1]
42 ABac[:,3]=vBac[2]
43 ABac[:,4]=vBac[3]
44 t4=pd.DataFrame(ABac,index=['|','|','|','|'],
45                  columns=['Eigenvalues','v1','v2','v3','v4'])
46 print(t4)

```

Table 1: Eigenvalues and Eigenvectors of VDI_{CB}

Eigenvalue	v_1	v_2	v_3	v_4	v_5
0.141	0.45	0.45	0.44	0.44	0.44
0.00124+0.00232i	-0.39+0.24i	0.10+0.0007i	-0.08-0.30i	0.52+0.24i	-0.59
0.00124-0.00232i	-0.39-0.24i	0.10-0.0007	-0.08+0.30i	0.52-0.24i	-0.59
0.00211	-0.98	0.01	-0.11	0.17	-0.06
0.00685	-0.15	-0.06	-0.20	0.94	-0.24

Table 2: Eigenvalues and Eigenvectors of BDI_{CB}

Eigenvalue	v_1	v_2	v_3	v_4	v_5
0.18	0.44	0.42	0.43	0.47	0.47
0.00959+0.00704i	0.19+0.04i	-0.08-0.12i	-0.02+0.11i	0.92	-0.04+0.27i
0.00959-0.00704i	0.19-0.04i	-0.08+0.12i	-0.02-0.11i	0.92	-0.04-0.27i
-0.00301	-0.59	-0.04	0.63	0.41	-0.31
0.00285	0.99	-0.01	0.07	-0.11	-0.01

Table 3: Eigenvalues and Eigenvectors of VDI_{AC}

Eigenvalue	v_1	v_2	v_3	v_4
0.211	0.50	0.49	0.51	0.50
0.0329	0.32	0.50	-0.39	-0.70
-0.0038	0.30	-0.02	0.43	-0.85
0.00248	0.36	-0.39	0.35	-0.77

Table 4: Eigenvalues and Eigenvectors of BDI_{AC}

Eigenvalue	v_1	v_2	v_3	v_4
0.297	-0.50	-0.46	-0.50	-0.54
0.0299	-0.28	-0.76	0.18	0.56
0.00348+0.00929i	-0.045+0.185i	0.84	-0.076+0.126i	0.040-0.481i
0.00348-0.00929i	-0.045-0.185i	0.84	-0.076-0.126i	0.040+0.481i

5) Norms of each row/column of each VDI/BDI matrix

```

1 # norm of each row/column of VDI/BDI
2
3 n=VDI_cb.shape[0]
4 print("\nRow infinity norms: Corbin Burnes VDI")
5 for i in range(n):
6     norm=max(abs(VDI_cb[i,:]))
7     print(f"Row {i}: {norm}")
8 print("\nColumn infinity norms: Corbin Burnes VDI")
9 for i in range(n):
10    norm=max(abs(VDI_cb[:,i]))
11    print(f"Column {i}: {norm}")
12 print("\nRow infinity norms: Corbin Burnes BDI")
13 for i in range(n):
14    norm=max(abs(BDI_cb[i,:]))
15    print(f"Row {i}: {norm}")
16 print("\nColumn infinity norms: Corbin Burnes BDI")
17 for i in range(n):
18    norm=max(abs(BDI_cb[:,i]))
19    print(f"Column {i}: {norm}")
20 n=VDI_ac.shape[0]
21 print("\nRow infinity norms: Aroldis Chapman VDI")
22 for i in range(n):
23    norm=max(abs(VDI_ac[i,:]))
24    print(f"Row {i}: {norm}")
25 print("\nColumn infinity norms: Aroldis Chapman VDI")
26 for i in range(n):
27    norm=max(abs(VDI_ac[:,i]))
28    print(f"Column {i}: {norm}")
29 print("\nRow infinity norms: Aroldis Chapman BDI")
30 for i in range(n):
31    norm=max(abs(BDI_ac[i,:]))
32    print(f"Row {i}: {norm}")
33 print("\nColumn infinity norms: Aroldis Chapman BDI")
34 for i in range(n):
35    norm=max(abs(BDI_ac[:,i]))
36    print(f"Column {i}: {norm}")

```

Row infinity norms: Corbin Burnes VDI

Row 1: 0.0846

Row 2: 0.0894

Row 3: 0.0888

Row 4: 0.0740

Row 5: 0.0878

Column infinity norms: Corbin Burnes VDI

Column 1: 0.0025

Column 2: 0.08943

Column 3: 0.0156
 Column 4: 0.0216
 Column 5: 0.0293
 Row infinity norms: Corbin Burnes BDI
 Row 1: 0.0955
 Row 2: 0.1036
 Row 3: 0.0966
 Row 4: 0.0862
 Row 5: 0.1012
 Column infinity norms: Corbin Burnes BDI
 Column 1: 0.0035
 Column 2: 0.1036
 Column 3: 0.0278
 Column 4: 0.0233
 Column 5: 0.0578
 Row infinity norms: Aroldis Chapman VDI
 Row 1: 0.1244
 Row 2: 0.121
 Row 3: 0.1058
 Row 4: 0.075
 Column infinity norms: Aroldis Chapman VDI
 Column 1: 0.1244
 Column 2: 0.0293
 Column 3: 0.0603
 Column 4: 0.0569
 Row infinity norms: Aroldis Chapman BDI
 Row 1: 0.18394
 Row 2: 0.1809
 Row 3: 0.1486
 Row 4: 0.1117
 Column infinity norms: Aroldis Chapman BDI
 Column 1: 0.1839
 Column 2: 0.0179
 Column 3: 0.1099
 Column 4: 0.0765

6) Modal Evolution tables/graphs

```

1 # Corbin Burnes (for BDI modes, sub BDI_cb into X1 and X2 for
  VDI_cb)
2
3 X1 = VDI_cb[:-1].T
4 X2 = VDI_cb[1:].T
5 A = X2 @ np.linalg.pinv(X1)
6 vals, vecs = np.linalg.eig(A)
7 x0 = X1[:, 0]
8 b = np.linalg.solve(vecs, x0)
9 T = VDI_cb.shape[0]
10 pitch_labels = ['FF', 'C', 'S', 'SL', 'CU']
11 num_modes = len(vals)
12 fig, axes = plt.subplots(num_modes, 1, figsize=(8, 3*num_modes),
  sharex=True)
13 if num_modes == 1:
14     axes = [axes]
15 for i in range(num_modes):
16     lambda_i = vals[i]
17     v_i = vecs[:, i]
18     b_i = b[i]
19     contribs = np.zeros((T, len(v_i)))
20     for t in range(T):
21         contribs[t, :] = np.real(b_i * (lambda_i ** t) * v_i)
22     df = pd.DataFrame(
23         contribs,
24         index=[f"{2021+t}" for t in range(T)],
25         columns=pitch_labels)
26     print(f"\n=====")
27     print(f"Mode {i+1} Contribution Table")
28     print("=====")
29     print(df.round(6))
30     ax = axes[i]
31     for j, label in enumerate(pitch_labels):
32         ax.plot(
33             range(T),
34             contribs[:, j],
35             marker='o',
36             label=label)
37     ax.set_title(f"Mode {i+1} contributions (all 5 pitches, Corbin
  Burnes: VDI)")
38     ax.set_ylabel("Contribution")
39     ax.grid(True)
40     ax.legend()
41 axes[-1].set_xlabel("Year index")
42 plt.tight_layout()
43 plt.show()

```

```

44
45 # Aroldis Chapman (for BDI modes, sub BDI_ac into X1 and X2 for
    VDI_ac)
46
47 X1 = VDI_ac[:-1].T
48 X2 = VDI_ac[1:].T
49 A = X2 @ np.linalg.pinv(X1)
50 vals, vecs = np.linalg.eig(A)
51 x0 = X1[:, 0]
52 b = np.linalg.solve(vecs, x0)
53 T = VDI_ac.shape[0]
54 pitch_labels = ['FF', 'SP', 'S', 'SL']
55 num_modes = len(vals)
56 fig, axes = plt.subplots(num_modes, 1, figsize=(8, 3*num_modes),
    sharex=True)
57 if num_modes == 1:
58     axes = [axes]
59 for i in range(num_modes):
60     lambda_i = vals[i]
61     v_i = vecs[:, i]
62     b_i = b[i]
63     contribs = np.zeros((T, len(v_i)))
64     for t in range(T):
65         contribs[t, :] = np.real(b_i * (lambda_i ** t) * v_i)
66     df = pd.DataFrame(
67         contribs,
68         index=[f"{2022+t}" for t in range(T)],
69         columns=pitch_labels)
70     print(f"\n=====")
71     print(f"Mode {i+1} Contribution Table")
72     print("=====")
73     print(df.round(6))
74     ax = axes[i]
75     for j, label in enumerate(pitch_labels):
76         ax.plot(
77             range(T),
78             contribs[:, j],
79             marker='o',
80             label=label)
81     ax.set_title(f"Mode {i+1} contributions (all 4 pitches, Aroldis
        Chapman: VDI)")
82     ax.set_ylabel("Contribution")
83     ax.grid(True)
84     ax.legend()
85 axes[-1].set_xlabel("Year index")
86 plt.tight_layout()
87 plt.show()

```

